

Tema 2

Ecuaciones lineales y No lineales

Metodo de Bisección

Se basa en aproximar raíces de $f(x)=0$

Requisitos:

- $f(x) = 0$
- Intervalo $[a, b]$ donde $f(a) \cdot f(b) < 0 \rightarrow$ cambio signo

Convergencia: Siempre converge

Proceso Iterativo:

1. Calcular Punto Medio $\rightarrow x_0 = \frac{a+b}{2}$

2. Nuevo Intervalo

- $f(a) \cdot f(x_0) < 0 \rightarrow$ Nuevo Intervalo $[a, x_0]$
- $f(b) \cdot f(x_0) < 0 \rightarrow$ Nuevo Intervalo $[x_0, b]$
- $f(x_0) = 0 \rightarrow$ Raíz

3. Criterio de Parada

- $|x_k - x_{k-1}| < \text{TOL} \rightarrow$ Parar cuando estemos lo suficientemente cerca de la raíz
- $|f(x_k)| < \text{TOL}_f \rightarrow$ Comprobar si la magnitud es cercana a 0
- $|b-a| < \text{TOL}_{\text{intervalo}} \rightarrow$ Detener cuando el intervalo sea pequeño

Algoritmo:

```
def biseccion(f,a,b, tol=1e-6, maxIter = 200, verbose=False):

    k = 0
    fa,fb = f(a),f(b)

    assert fa*fb < 0, "No se puede aplicar biseccion"

    if verbose:
        print(f"{k:>3} {a:>12} {b:>12} {midpoint:>12} {f(mid):>12} {error:>12}")

    while k < maxIter:

        midpoint = (a + b)/2
        f_midpoint = f(midpoint)

        error = np.abs(b - a)

        if verbose:
            print(f"{k:3d} {a:12.6e} {b:12.6e} {midpoint:12.6e} {f_midpoint:12.6e} {error:12.6e}")

        if fa*f_midpoint < 0:
            b, fb = midpoint, f_midpoint

        elif fb*f_midpoint < 0:
            a, fa = midpoint, f_midpoint

        else:
            if verbose:
                print(f"\nRaíz exacta encontrada en x = {midpoint}")
            break

        if error < tol:
            break

        k += 1

    else:
        print("Max Iter Reached")

    return midpoint
```

Convergencia

Quiero saber cuán rápido convergen las aproximaciones a la solución

Criterio de convergencia: $\lim_{k \rightarrow \infty} |e_k| = 0$ con $e_k = x_k - x^*$ → Error de Aproximación

Numero Iteraciones: $k \geq N = \left\lceil \frac{\ln(b-a) - \ln(2\varepsilon)}{\ln(2)} \right\rceil$ con $|e_k| = |x_k - x^*| \leq \varepsilon$

Orden de Convergencia

Queremos verificar cuán rápido el método se acerca a x^*

la velocidad de convergencia se mide mediante el orden de convergencia (r).

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|^r} = C \rightarrow r > 1 \rightarrow C > 0$$

$$|e_{k+1}| \approx C |e_k|^r < |e_k|$$

Tipos de Convergencia

- Convergencia Lineal: ($r=1$): $|e_{k+1}| \approx C |e_k|$ con $C < 1$

- Convergencia Cuadrática ($r=2$): $|e_{k+1}| \approx C |e_k|^2$

Convergencia en el Método de Bisección: Convergencia lineal con $C = 1/2$

Pasos:

- $|e_2| \approx C |e_1|^r$ y $|e_3| \approx C |e_2|^r$

- $\frac{|e_2|}{|e_3|} \approx \left(\frac{|e_1|}{|e_2|} \right)^r \rightarrow r \approx \frac{\ln\left(\frac{|e_2|}{|e_3|}\right)}{\ln\left(\frac{|e_1|}{|e_2|}\right)}$

Código:

```
def biseccion(f,a,b,tol=1.0e-6,maxit=200,verbose=False):  
  
    fa,fb = f(a),f(b)  
  
    assert fa*fb < 0, "No se cumplen las condiciones para aplicar M. de Biseccion"  
    errores = []  
    ratio = -1  
  
    for k in range(0,maxit):  
        c = (a+b)/2  
        fc = f(c)  
  
        if fa * fc < 0:  
            b, fb = c, fc  
        elif fb * fc < 0:  
            a, fa = c, fc  
        else:  
            break  
  
        error = b - a  
        errores.append(error)  
  
        if (len(errores) > 3):  
            errores.pop(0)  
            e1 = errores[-3]  
            e2 = errores[-2]  
            e3 = errores[-1]  
            ratio = np.log(e2/e3)/np.log(e1/e2)  
  
        if verbose:  
            print(f"Iteracion: {k}, a={a}, b={b}, c={c}, f(c)={fc}, ratio={ratio:e}")  
  
        if np.abs(error) < tol:  
            break  
  
    else: #Solo ocurre cuando se completa todo el bucle  
        print(f"Maximo de Iteraciones {maxit} alcanzadas")  
  
    return c
```

Iteración de Punto Fijo

Punto Fijo: Punto x^* tal que $g(x^*) = x^*$

Aproxima puntos fijos de la función $g(x)$

Idea: Rescribir $f(x) = 0$ en la forma $x = g(x)$

Ejemplo: $f(x) = \sin(x) + 1 = 0$

- Forma 1 $\rightarrow x = x + 1 + \sin(x) \rightarrow$ Sumamos x a ambos lados

- Forma 2 $\rightarrow -1 = \sin(x) \rightarrow x = -x \sin(x)$

La elección de $g(x)$ es crucial para la convergencia y eficiencia

Proceso Iterativo:

1. Dado $x = g(x) \rightarrow x_{k+1} = g(x_k)$

2. Partimos de x_0

3. Calcular Iterativamente: $x_1 = g(x_0)$, $x_2 = g(x_1)$, ...

4. Si la secuencia $\{x_k\}_{k \in \mathbb{N}}$ converge, esperamos que el límite sea solución $x^* = g(x^*)$

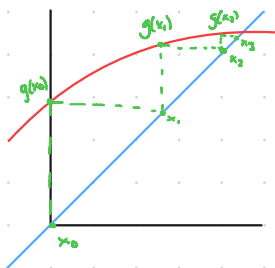
Implementación:

```
def punto_fijo(g, x0, tol=1.e-6, maxit=200, verbose=False):  
  
    k = 0  
    x_k = x0  
    x_k_minus1 = x0  
  
    print(f"{'k':>4} {'x_k':>18} {'cota error':>14}")  
    print("-" * 42)  
  
    while k < maxit:  
  
        x_k = g(x_k_minus1)  
        error = np.abs(x_k - x_k_minus1)  
  
        if verbose:  
            print(f"{'k':>4} {'x_k':>18.10f} {'error':>14e}")  
  
        if error < tol:          # Criterio de parada  
            break  
  
        k += 1  
    else:  
        print(f"\nAdvertencia: número máximo de iteraciones ({maxit}) alcanzado.")  
  
    return x_k
```

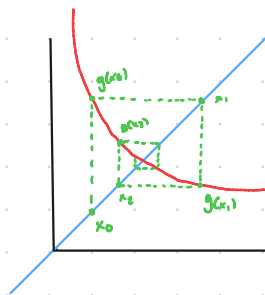
Interpretación Geométrica:

La iteración de punto fijo se visualiza si converge como una escalera o telaraña.

Escalera:



Telaraña:



Pasos:

1. Partiendo de x_0 , subimos hasta $y = g(x)$ para obtener $g(x_0) = x_1$.
2. No movemos horizontalmente hasta el punto (x_1, x_1) .
3. Repetir

Nota: Convergencia depende de $g(x)$ y x_0

Convergencia:

Orden 1 $\rightarrow$ Convergencia lineal

Si $g'(x^*) = 0 \rightarrow$ Convergencia superlineal $\rightarrow r > 1$

La condición $|g'(x)| \leq K < 1$ en un entorno x^* es suficiente para garantizar la convergencia local del método

Se puede clasificar al punto fijo de 2 maneras:

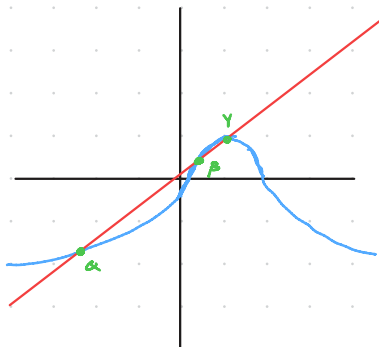
- Si $|g'(x^*)| < 1 \rightarrow$ Atractor
- Si $|g'(x^*)| > 1 \rightarrow$ Repulsor $\rightarrow$ No converge hacia punto fijo, solo puntos sobre recta $y =$ punto repulsor
- Si $|g'(x^*)| = 1 \rightarrow$ Depende de derivadas superiores

Ejemplo:

$$f(x) = -x - 3 + 4 \operatorname{sech}(x-1)$$

$$g(x) = -3 + 4 \operatorname{sech}(x-1)$$

$$g'(x) = -4 \operatorname{sech}(x-1) \tanh(x-1)$$



Punto $\alpha \rightarrow \alpha \approx -2,8257 \rightarrow |g'(\alpha)| < 1 \rightarrow$ Atractor

Punto $\beta \rightarrow \beta \approx 0,4327 \rightarrow |g'(\beta)| > 1 \rightarrow$ Repulsor

Punto $\gamma \rightarrow \gamma = 1 \rightarrow |g'(\gamma)| = 0 \rightarrow$ Atractor

Método de Newton

Método de iteración de punto fijo con convergencia cuadrática

Se aproxima $f(x)$ mediante su recta tangente en un punto $(x_k, f(x_k))$

Ecuación Recta Tangente: $y = f'(x_k)x + b$ con $b = f(x_k) - f'(x_k)x_k$

$$y = f'(x_k)(x - x_k) + f(x_k)$$

Iteración de Newton: $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$

Implementación:

```
def newton(f, df, x0, tol=1e-6, maxit=100, verbose=True):  
  
    x_k = x0  
    x_kplus1 = x0  
    error = 0  
  
    if verbose:  
        print("k\t\tx_k\t\tx_k+1\t\terror")  
  
    for k in range(maxit):  
  
        f_xk = f(x_k)  
        df_xk = df(x_k)  
  
        if f_xk != 0:  
            x_kplus1 = x_k - (f_xk/df_xk)  
  
            error = np.abs(x_kplus1 - x_k)  
  
            if verbose:  
                print(f"{k}\t\t{x_k}\t\t{x_kplus1}\t\t{error}")  
  
            if error < tol:  
                break  
  
        x_k = x_kplus1  
  
    else:  
        print("Numero Maximo de Iteraciones Alcanzado")  
  
    return x_kplus1
```

Newton como Iteración de Punto Fijo :

$$x_{k+1} = g(x_k) \quad \text{con} \quad g(x) = x - \frac{f(x)}{f'(x)}$$

Si f es una función con 2 derivadas en un entorno :

$$g'(x) = 1 - \frac{(f'(x))^2 - f(x)f''(x)}{(f'(x))^2} = \frac{f(x)f''(x)}{(f'(x))^2}$$

Si $f'(x) \neq 0 \rightarrow g'(x) = 0 \rightarrow$ Convergencia $r \geq 1$

Convergencia: Convergencia Cuadrática $\rightarrow \lim_{k \rightarrow \infty} \frac{e_{k+1}}{e_k^2} = \frac{f''(x^*)}{2f'(x^*)}$

Método de la Secante

Alternativa al método de Newton $\rightarrow$ Evita cálculo de derivadas

Aproximación a derivadas con diferencias finitas : $f'(x_n) = \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$

Aproximación mediante línea secante que pasa por $(x_{n-1}, f(x_{n-1}))$ y $(x_n, f(x_n))$

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} = \frac{f(x_n)x_{n-1} - f(x_{n-1})x_n}{f(x_n) - f(x_{n-1})}$$

Implementación:

```
def secante(f, x1, x2, tol=1e-6, maxit=100, verbose=True):  
  
    x_kminus1 = x1  
    x_k = x2  
    x_kplus1 = x2  
    error = 0  
  
    if verbose:  
        print("k\t\tx_k\t\tx_{k+1}\t\terror")  
  
    for k in range(maxit):  
  
        f_xk = f(x_k)  
        f_xkminus1 = f(x_kminus1)  
  
        x_kplus1 = x_k - f_xk * ((x_k - x_kminus1) / (f_xk - f_xkminus1))  
  
        error = np.abs(x_kplus1 - x_k)  
  
        if verbose:  
            print(f"{k}\t\t{x_k}\t\t{x_kplus1}\t\t{error}")  
  
        if error < tol:  
            break  
  
        x_kminus1 = x_k  
        x_k = x_kplus1  
  
    else:  
        print("Numero Maximo de Iteraciones Alcanzado")  
  
    return x_kplus1
```

Orden de Convergencia: Número áureo $\rightarrow r = \frac{1+\sqrt{5}}{2} \approx 1,618$